1. Which dependencies did you use and why?

In the posts Manager applications, I used three main dependencies.

1. http package.

The http package is used to send request to the API. It allow the application to communicate with the server.

2. dart: Convert

dart: convert is built-in Dart library used for JSON convertion it helps convert data between JSON format and Dart objects. Main function used are jsonDecode() which convert JSON Data into Dart objects and jsonEncode which converts Dart object into JSON

3. Dart:io: Is used to to detect internet connection problems

This prevents the applications from crashing and allows it to show friendly error message.

⚠ Dart: convert and Dart: io is built in library

2. Generally, how can you hand API exception (Network errors, invalid data)?

API calls can fails because of many reasons such as network problems, server errors or invalid responses. Therefore, error handling is very important

① Use try-catch blocks

API request should be wrapped inside a try-catch block to capture possible errors.

② Handle different types of errors

Different exception should be handled separately so we can provide to user friendly message when that error occurs.

here are examples which is better to handle it separately such as NetworkException, ServerException, NotFoundException and ParseException

These custom exceptions help the App to understand what went wrong.

3. Checking HTTP status codes.

After receiving the response, the application should check the status code

Examples:

200/201 : Request successful

404: Resource not found

400–499 : Client error

500+ : Server error

4. Show user_friendly message in the UI.

Errors should be displayed to the user in clear way. This improves the user experience.

3. Explain the Future widget you used and which part of the UI depends on it.

What is Future?

In Dart, a Future ~~request~~ present a value that will be available later. it is used for asynchronous operations such as API request.

What is FutureBuilder?

FutureBuilder is a flutter widget that listens to a Future and rebuilds the UI when the Future changes state

here are three states of futureBuilder

① Waiting: API request still running

② Error state: API request failed

③ Done: The API request succeded.

Which part of UI depends on the Future?

In this application, The UI elements that depend on the Future are:

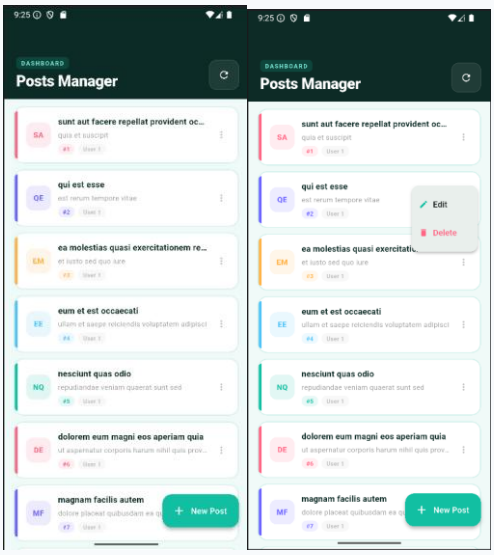
- Post list screen: displays posts from the API
- Post detail screen: display the details of a selected post
- Loading indicator: Shown while waiting for the API response
- Error display: shown if the API requests fails.

App Screenshots

Post List Screen (Home Screen)

Shows all posts fetched from GET /posts displayed as a scrollable list. Each card shows the post ID, user ID, title, and body preview.

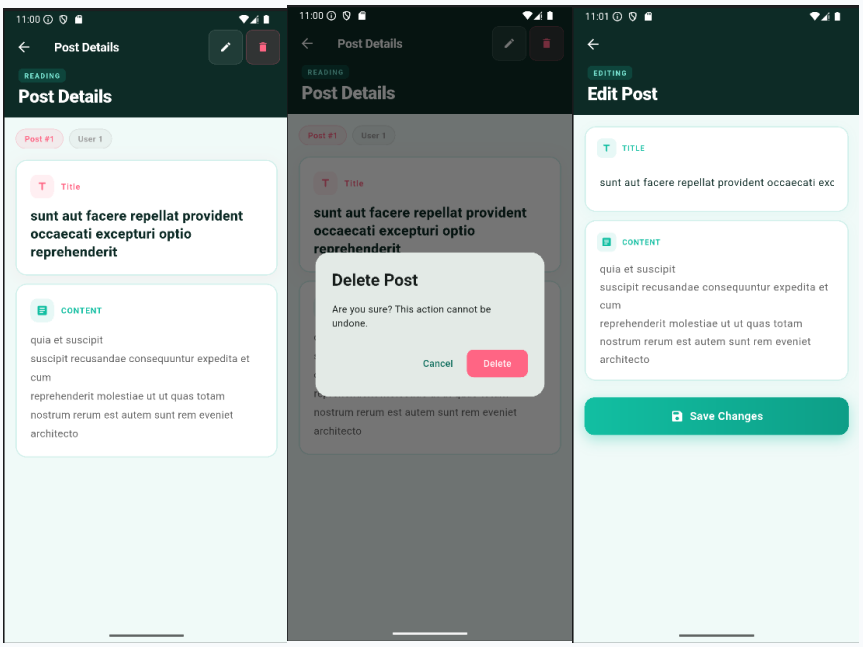
Screenshot — Post List Screen



Post Detail Screen

Shows the full content of a single post fetched from GET /posts/{id}. Tap any post in the list to navigate here.

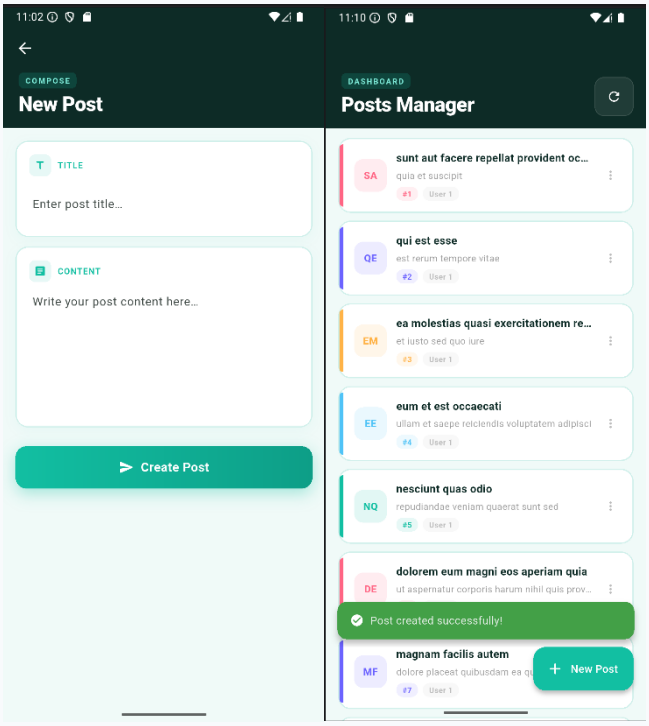
Screenshot — Post Detail Screen



Create New Post Screen

Empty form for creating a new post. Opened by tapping the 'New Post' floating button. Submits a POST request to the API.

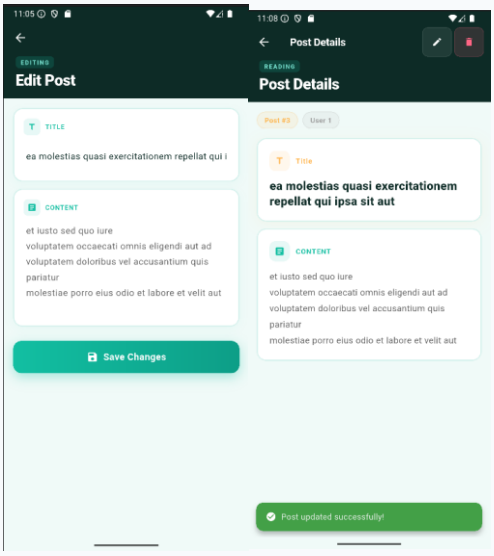
Screenshot — Create Post Form (Empty) and success message



Edit Post Screen

Same form as Create, but pre-filled with the selected post's existing data. Opened via the 3-dot menu on a post card. Sends a PUT request.

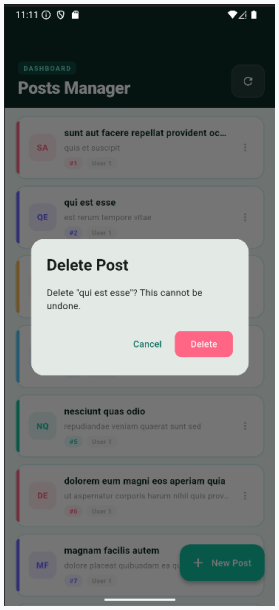
Screenshot Edit Post Form (Pre-filled) and success message



Delete Confirmation Dialog

Alert dialog asking the user to confirm deletion. Triggered from the 3-dot menu on any post card. On confirmation, sends DELETE /posts/{id}.

Screenshot — Delete Confirmation Dialog



Code Screenshot 1 — Dependencies (Question 1)

pubspec.yaml (Dependency Declaration)

File location: posts_manager/pubspec.yaml

What it proves: The http package is the only external dependency added. dart:convert and dart:io are Dart SDK built-ins requiring no entry here.

Code Screenshot Q1 — pubspec.yaml showing http: ^1.2.0

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0
```

Code Screenshot 2 — Exception Handling (Question 2)

post_service.dart (try/catch Exception Block)

File location: lib/services/post_service.dart

What it proves: The app handles network failures, timeouts, bad JSON, and HTTP errors gracefully using a custom exception hierarchy.

Code Screenshot Q2 — try/catch block in post_service.dart

```
try {
  await _service.deletePost(post.id!);
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: const Row(children: [
        Icon(Icons.check_circle_rounded, color: Colors.white, size: 18),
        SizedBox(width: 8),
        Text('Post deleted'),
      ]), // Row
      backgroundColor: Colors.green.shade600,
      behavior: SnackBarBehavior.floating,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
    ), // SnackBar
  );
  _loadPosts();
} on ApiException catch (e) {
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Delete failed: ${e.message}'), behavior: SnackBarBehavior.floating),
  );
}
```

Code Screenshot 3 — FutureBuilder Widget (Question 3)

post_list_screen.dart (FutureBuilder Widget)

File location: lib/screens/post_list_screen.dart

What it proves: FutureBuilder automatically manages all 3 async UI states (loading, error, success) without manual setState() calls.

Code Screenshot D1 — FutureBuilder in post_list_screen.dart

```
body: FutureBuilder<List<Post>>(        You, 2 hours ago • Uncommitted changes
  future: _postsFuture,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            CircularProgressIndicator(color: _kPrimary),
            SizedBox(height: 16),
            Text('Loading posts…', style: TextStyle(color: Colors.grey)),
          ],
        ), // Column
      ); // Center
    }
```